

The Ensemble Idea

A single decision tree is a **high-variance** model, it is **highly sensitive** to the **specific training data** it sees, and a **fully grown tree** **memorizes noise**. The solution is not to build a better individual tree, but to build **many trees** and **combine their predictions**.

This is the **ensemble idea**, and it is one of the most powerful and practically important concepts in machine learning.

There are **two fundamentally different strategies**:

- **Bagging**

This is short for **Bootstrap Aggregating**. It **builds trees in parallel**, each on a **different random subset** of the data, and **averages their predictions**. The goal is **variance reduction**. Each tree overfits slightly differently, and averaging smooths out the idiosyncratic errors.

- **Boosting**

Builds trees **sequentially**, each one correcting the errors of the previous ensemble. The goal is **bias reduction**, each new tree focuses on the examples the current ensemble **gets wrong**.

Random Forests

Bootstrap Aggregating

The **bootstrap** is a **statistical technique** for sampling data to evaluate models.

Suppose you've trained a model on a dataset and you want to know how good it is, its **generalization error**. Ideally you'd evaluate it on **fresh data**

drawn from the same distribution.

However, you only have **one dataset**.

You could separate a chunk of that into a **test set**, but that reduces the data for training, and the estimate of error depends on which specific split you chose.

The bootstrap asks: can we use the **single dataset** to **simulate** having drawn **many different datasets** from the **same distribution**?

The key insight is that your training dataset is itself a sample from some unknown distribution. The **empirical distribution** of your data, the distribution that puts **equal probability $1/n$** on each training example, is the best estimate you have of the true underlying distribution, since we **can't access the true distribution** (we don't know it). So we sample from the empirical distribution **with replacement**.

So, given a dataset D of n examples, a **bootstrap sample D^*** is created by drawing n examples from D with replacement. Because of this, some examples will appear multiple times, and some won't appear at all.

On average, each bootstrap sample contains about **63.2%** of the unique original examples. The other **36.8%** of examples that weren't selected for D^* are called **out-of-bag (OOB)** samples.

They are **unused** for a model trained on D^* - which makes them a natural validation set. Thus, OOB samples make a **natural validation set**.

The Bagging Algorithm

Bagging trains **B trees**, each on a

different bootstrap sample. So:

- for $b = 1, 2, \dots, B$
- Draw a bootstrap sample D_b of size n from the training data with replacement
- Train a full decision tree T_b on D_b .
- For classification - pick the majority vote across all B trees
- For regression - predict by averaging the B tree predictions.

The final prediction for a new point x is:

$$\hat{y} = \frac{1}{B} \sum_b T_b(x) \quad (\text{regression})$$

$$\hat{y} = \text{majority_vote} \{T_b(x)\} \quad (\text{classification})$$

Why Bagging Reduces Variance: The Maths

Suppose we have B models $T_1, T_2, \dots, T_B$. Each has the same variance σ^2 (they're all trained the same way on similarly sized datasets.) Any two trees have a pairwise correlation ρ . This measures how similarly they behave across new datapoints. Specifically, it is the Pearson correlation between their predictions:

$$\rho = \frac{\text{Cov}(T_b, T_{b'})}{\sigma^2}$$

If you feed 1000 new data points to both trees and compared their predictions, ρ measures how linearly related those prediction vectors are. $\rho = 1$ means the trees always make identical predictions. $\rho = 0$ means their predictions are completely unrelated.

Trees trained on similar data using similar features tend to make similar

splits near the root and therefore have high ρ . Trees that see different data and are forced to use different features have lower ρ .

Returning to the model, our ensemble prediction is the average:

$$T = \frac{1}{B} \sum_b T_b$$

We want to find $\text{Var}(T)$. Recall the identity for the Variance of a sum:

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$$

For a sum of B variables, this generalises to

$$\text{Var}\left(\sum_b T_b\right) = \sum_b \text{Var}(T_b) + \sum_b \sum_{b' \neq b} \text{Cov}(T_b, T_{b'})$$

The first term sums B variances, each equal to σ^2 .

$$\sum_b \text{Var}(T_b) = B\sigma^2$$

The second term sums all pairwise covariances. There are $B(B-1)$ ordered pairs (b, b') with $b \neq b'$. The Covariance between two trees with correlation ρ and variance σ^2 is:

$$\begin{aligned} \text{Cov}(T_b, T_{b'}) &= \rho \times \sigma \times \sigma \\ &= \rho \sigma^2 \end{aligned}$$

So

$$\sum_b \sum_{b' \neq b} \text{Cov}(T_b, T_{b'}) = B(B-1)\rho\sigma^2$$

Therefore:

$$\text{Var}\left(\sum_b T_b\right) = B\sigma^2 + B(B-1)\rho\sigma^2$$

Since $T = \frac{1}{B} \sum_b T_b$ and variance scales

with the square of the constant:

$$\text{Var}(T) = \frac{1}{B^2} \cdot \text{Var}\left(\sum_b T_b\right)$$

$$= \frac{1}{B^2} \cdot [B\sigma^2 + B(B-1)\rho\sigma^2]$$

$$= \frac{\sigma^2}{B} + \frac{(B-1)\rho\sigma^2}{B}$$

$$= \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{B}$$

This has two terms with completely different characters:

- $\rho\sigma^2$

Does not depend on B at all. No matter how many trees you add, this term stays fixed. It is the **irreducible variance floor** - the minimum variance the ensemble can achieve regardless of how large B gets.

- $(1-\rho)\sigma^2/B$

Decreases as B increases. Adding more trees always shrinks this term. As $B \rightarrow \infty$ it vanishes completely.

The formula reveals two completely independent ways to reduce ensemble variance:

- **Increase B**

Add more trees. This reduces the second term. The returns are diminishing, going from 1 to 10 trees helps much more than going from 100 to 110.

- **Decrease ρ**

Make the trees less correlated. This reduces both terms simultaneously. It lowers the floor $\rho\sigma^2$ and makes the second term smaller too.

The limiting cases:

- $\rho = 0$ (perfectly independent trees)

$$\text{Var}(T) = \frac{\sigma^2}{B}$$

Variance shrinks all the way to 0 as $B \rightarrow \infty$. Perfect variance reduction.

- $\rho = 1$ (perfectly identical trees)

$$\text{Var}(T) = \sigma^2$$

Averaging identical trees gives you nothing. The ensemble has the same variance as each individual tree. This is why correlation between trees is the enemy of bagging.

Feature Randomisation (Random Forests)

Standard bagging builds B trees on bootstrap samples of the data. The problem is, the trees are **highly correlated**. If there is one very strong predictor, almost every tree will use it near the root, and the trees will look similar to each other. High correlation ρ means the variance floor remains large.

Random Forests decorrelate the trees by adding **randomness** at the feature level.

At each node split, instead of considering all d features, only a **random subset** $m < d$ features is considered. The best split is chosen from only these m features. A new random subset is drawn at every node independently.

Choosing m

The size of feature subset m is the primary hyperparameter of Random Forests:

- **Classification**

$m = \sqrt{d}$ is the standard default

- **Regression**

$m = d/3$ is the standard default

smaller $m \rightarrow$ more randomisation $\rightarrow$

lower correlation between trees $\rightarrow$ lower variance floor, but each individual tree is weaker.

Why Feature Randomisation Works

By restricting each split to m features, we prevent the dominant features from always being chosen. Trees are forced to explore different features at different nodes, making them structurally different from each other. Their predictions are therefore less correlated, and the variance floor drops.

The individual trees are slightly weaker (higher bias), because they sometimes miss the best feature. However, the ensemble is much stronger because the variance reduction from lower correlation more than compensates for the slight bias increase.

Part II: Gradient Boosting

Bagging builds trees in parallel on random subsets of data to reduce variance.

Boosting builds trees sequentially, each one targeting the errors of the previous ensemble. The goal of boosting is bias reduction, making a strong learner from many weak learners, each incrementally improving the ensemble's predictions.

The original boosting algorithm, **Adaboost**, reweights training examples at each step, giving more weight to misclassified examples. Gradient Boosting takes a more general approach: It performs gradient descent on the function itself. This concept is best explained through example.

Consider standard Gradient Descent. We have a loss function $\mathcal{L}(w)$ that depends on a fixed set of parameters w . We minimise it by repeatedly nudging w in the direction that reduces $\mathcal{L}$ the most:

$$w \leftarrow w - \eta \cdot \nabla_w \mathcal{L}$$

The gradient $\nabla_w \mathcal{L}$ tells us how much the loss changes when we change each parameter.

Gradient Boosting asks a different question. Instead of a fixed parameter vector w , what if our model is an abstract function F , a mapping from inputs x to predictions $F(x)$? In the linear / logistic regression etc. case, this function is defined purely by that fixed parameter vector, so we can perform gradient descent as usual. But for trees there are no such parameters.

Instead we're optimising over the space of all possible functions F . We want to find the function F that minimises the total loss over all training examples. This is a much larger and more abstract space.

So what is a gradient when you're optimising a function rather than a vector? Let's build up the intuition.

For each training example i , F makes a prediction $F(x_i)$. That prediction is just a scalar. The loss $\mathcal{L}(y_i, F(x_i))$ depends on that number. We can compute the per-input loss:

$$r_i = - \frac{\partial}{\partial F(x_i)} \mathcal{L}(y_i, F(x_i))$$

Telling us which direction you could change the prediction at i to reduce the loss the most. We do this for all inputs and stack them into a vector $r = (r_1, r_2, \dots, r_n)$. So now we have the direction in prediction space that reduces the loss most steeply. A functional analogue to the gradient.

However, we have an issue: generalisation. The vector r only tells you how to adjust the predictions from the training points you already have. But it has no meaning for any point outside the training set. If we add a new point x_j , there is no corresponding loss r_j . r is only a vector of adjustments to specific training predictions, not a function that generalises.

To solve this, for each new point x_i , we predict its corresponding r_i via regression. We need a regression model h that takes any input x , including ones never seen during training and outputs the appropriate adjustment. Formally, we need:

$$h(x_i) \approx r_i \quad \text{for all training points } i.$$

So, we train a regression tree on all pairs (x_i, r_i) to do this. It learns the mapping from features to adjustment.

So our function update rule is:

$$F_{\text{new}}(x) = F_{\text{old}}(x) + \eta \cdot h(x)$$

where h is the new tree. For every new input x , it tells us approximately how to adjust the prediction in the direction that reduces the loss.

Why use a tree and not a regular regression model? In theory, you could, but in practice we always use a regression tree for a few key reasons:

- **Non-linearity**

The residuals often have complex non-linear structure that a linear regression model wouldn't accurately capture.

- **No feature scaling needed**

Linear models would be sensitive to feature scales, requiring standardisation at every learning step.

- **Computational Efficiency**

Trees are fast to fit, linear models are expensive.

In short, trees are fast and flexible. They're a natural choice.

The Algorithm

1. Initialisation

Before we have any trees, we need some starting prediction for every training point. Since we've yet to learn anything our best guess is just the mean of the targets.

$$F_0(x) = \bar{y}$$

This is the starting ensemble. Every training point gets the same prediction, the overall mean.

2. Compute pseudo-residuals

For each training example i , compute how wrong the current ensemble is, and in which direction it should move:

$$r_i = -\frac{\partial l(y_i, F(x_i))}{\partial F(x_i)}$$

For MSE loss, this is simply $y_i - F(x_i)$ - the true residual.

This all together gives a vector of n numbers, one for each training point, telling you the direction and magnitude of the desired adjustment at each point.

3. Fit a regression tree to the pseudo-residuals

Train a shallow regression tree on the dataset $\{(x^{(1)}, r_1), (x^{(2)}, r_2), \dots, (x^{(n)}, r_n)\}$.

The inputs are the original features, the targets are the pseudo-residuals.

The tree learns to map features to the desired adjustment. It doesn't need to fit perfectly - it just needs to capture the main patterns in how the pseudo-residuals relate to the inputs.

3. Update the ensemble

$$F_{new}(x) = F_{old}(x) + \eta \cdot h(x)$$

Note that this is done **element wise**. Each prediction is individually shifted by the step the pseudo-residuals are pointed. x there is the vector of all training points, not one individual one.

4. Repeat

After T steps, the cumulative picture is:

$$F_T(x) = F_0 + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \dots + \eta \cdot h_T(x)$$

The learning rate $\eta \in [0, 1]$ scales the contribution of each tree. Small η means each tree makes a small correction - the ensemble converges slowly but more carefully, typically achieving better generalisation. Large η means each tree makes a large correction - fast convergence but risk of overshooting and overfitting.

The learning rate and number of trees T are tightly coupled: small η requires large T to achieve the same training loss.

Gradient boosting trees are typically very **shallow**, depth 3 to 6. This is the opposite of Random Forests, which use deep trees. Gradient boosting reduces bias **sequentially** - each tree only needs to make a small correction. Shallow trees have **lower variance** and are faster to fit.

The ensemble of many shallow trees achieves low bias through their **combination**.

Regularisation in Gradient Boosting

Subsampling (Stochastic Gradient Boosting)

At each boosting step, instead of using all n training examples, we use a **random subsample** fraction $f \in [0, 1]$. This is called **stochastic gradient boosting**. It serves two purposes: it **reduces computation per step**, and it **introduces randomness** that **decorrelates the trees**, reducing overfitting.

Similar to how we did the feature randomisation in Random Forests. Typical values are $\gamma = 0.5$ to 0.8 . Using $\gamma < 1$ almost always improves generalisation.

Feature Subsampling

As in Random Forests, we can randomly select a subset of features at each tree or at each node. This further decorrelates the tree and reduces overfitting.

Early Stopping

With a validation set, we monitor the validation loss at each boosting step. Training stops when the validation loss has not improved for a specified number of rounds. This automatically determines the optimal T without requiring a grid search, and prevents overfitting to the training data.

Early stopping is the single most important regularisation technique for gradient boosting in practice. Always use it.

XGBoost: Second-Order Optimisation

Standard Gradient Boosting fits each tree to the pseudo-residuals - the first-order gradient of the loss. This tells you the **direction** to move predictions, but not **how far**. We apply a **fixed learning rate** η to every tree uniformly.

It doesn't account for the fact that some predictions are in flatter regions of the loss (where you could take a bigger step) and some are in steeper regions (where you should be more careful).

This is analogous to **standard gradient descent** ignoring curvature - Newton's method uses the **second derivative** to adapt the step

to local geometry, and converges much faster. XGBoost does the same thing for gradient boosting. It uses a **second-order Taylor expansion** of the loss around the current predictions. Instead of just computing the **gradient**

$$g_i = \frac{\partial L}{\partial F(x_i)}$$

it also computes the **Hessian**

$$h = \frac{\partial^2 L}{\partial F(x_i)^2}$$

The curvature of the loss at each prediction. It tells you how **quickly the gradient is changing**. High curvature means the gradient changes fast, thus, take a small step. Similar logic for low curvature.

So when we're building the tree for pseudo-residuals, in standard gradient boosting each leaf node is just the mean of the pseudo-residuals in that leaf. In XGBoost, the optimal leaf value is **derived analytically** using both the gradients and Hessians.

$$\omega^* = - \frac{\sum G}{\sum H + \lambda}$$

where $\sum G$ is the **sum of gradients** and $\sum H$ is the **sum of Hessians**. The λ is an **L2 regularisation term** - it prevents leaf values from becoming too large. Note that if the Hessian is large, the denominator is larger and the leaf value is smaller - the model takes a more conservative step. If the Hessian is small, the leaf value is larger and a bigger step is taken.

